

Week 6 — Spark Architecture & Efficient Data Processing

Code with Execution Outputs

Objective: Explore how Spark is structured internally and use it to process data efficiently through transformations, filtering, schema control, and optimized storage formats.

Dataset: orders_data.csv

The notebook moves through the objective one step at a time:

1. How Spark is organized (Driver, Cluster Manager, Executors) and its execution modes
2. Lazy evaluation and the DAG (lineage graph)
3. Loading files (CSV, Parquet) while controlling the schema
4. Selecting and filtering the columns that matter
5. Reshaping DataFrames (renaming, casting, adding columns)
6. Distinguishing transformations from actions
7. Wide transformations and performance factors (shuffle, predicate pushdown)
8. Comparing CSV against Parquet and what it means for speed
9. Dealing with missing values and filtering cleanly
10. Assembling a pipeline (read → transform → filter → write)
11. Writing results back out as CSV and Parquet
12. Practical habits for big data (favor show() over collect())

Every section pairs PySpark code with its output and a short takeaway.

1. Spark Architecture & Execution Modes

Main building blocks:

- **Driver** : This is the central process behind a Spark program. It sets up the SparkSession, holds the SparkContext, turns your operations into an execution plan (the DAG), breaks that plan into stages and tasks, and hands them out for processing. Once the executors finish, the driver gathers their results.
- **Cluster Manager** : This component hands out compute resources (CPU and memory) across the machines in the cluster. Whenever the driver needs capacity, the cluster manager spins up executors on worker nodes. Spark can work with several of them, such as Standalone, YARN, Apache Mesos, and Kubernetes.
- **Executors** : These are JVM processes that live on the worker machines. They carry out the tasks the driver sends them, keep data partitions in memory or on disk, and return the computed results. How many tasks an executor can run at once depends on the CPU cores it has been given.

Execution modes:

- **Client mode** : Here the driver runs on whatever machine launched the job, for example a laptop or an edge node. It suits development and interactive exploration, but because the driver stays on that machine, losing the connection ends the application.
- **Cluster mode** : Here the driver is placed on a worker node inside the cluster itself. That decouples the job from the launching machine, which is sturdier and generally the choice for production.

The cell below queries the running session so these ideas map onto something concrete.

Code:

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.functions import col
from pyspark.sql.types import (StructType, StructField, StringType,
                                DoubleType, IntegerType)

# creating a spark session
spark = (SparkSession.builder
        .appName("Week6_Spark_Architecture")    # name of the application
        .master("local[*]")    # run locally with all available cores
        .getOrCreate())

# Show only error logs
spark.sparkContext.setLogLevel("ERROR")

# get SparkContext
sc = spark.sparkContext
print("Spark version :", spark.version)
print("Application name :", sc.appName)
print("Master :", sc.master)
```

Output:

```
Spark version : 3.5.1
Application name : Week6_Spark_Architecture
Master : local[*]
```

2. Lazy Evaluation & the DAG (Lineage Graph)

Spark does not act on operations the moment you write them. Calls like `filter`, `select`, and `withColumn` are simply noted down as a chain of steps, captured in a Directed Acyclic Graph (DAG) — the lineage — that describes the transformations to be done.

Nothing actually runs until an action like `show()`, `count()`, or `write()` is invoked. Only then does Spark examine the full plan, refine it with the Catalyst optimizer, and finally compute the data.

Why this helps performance: seeing the whole chain up front lets Spark fold narrow steps into one pass over the data, ignore columns that are never needed, and avoid saving intermediate results it can skip.

The code below illustrates this deferred execution and the resulting DAG :

Code:

```
# Build a chain of transformations (executed lazily)
lazy_df = (spark.read.csv("../data/orders_data.csv", header=True,
inferSchema=True)
            .filter(col("category") == "Electronics")
            .select("product_id", "price"))

# At this stage, Spark has only created the execution plan (DAG)
print("Transformations defined, but no job has executed yet.")

# The action below is what actually triggers execution
print("Row count (action triggers the DAG):", lazy_df.count())
```

Output:

```
Transformations defined, but no job has executed yet.
Row count (action triggers the DAG): 1363
```

Code:

```
# View the logical and physical execution plan
lazy_df.explain(mode="formatted")
```

Output:

```
== Physical Plan ==
* Project (3)
+- * Filter (2)
   +- Scan csv (1)

(1) Scan csv
Output [3]: [product_id#1057, category#1058, price#1059]
Batched: false
Location: InMemoryFileIndex [file:/d:/CEI internship/Week
6/Spark_Assignment/data/orders_data.csv]
PushedFilters: [IsNotNull(category), EqualTo(category,Electronics)]
ReadSchema: struct<product_id:string,category:string,price:double>

(2) Filter [codegen id : 1]
Input [3]: [product_id#1057, category#1058, price#1059]
Condition : (isnotnull(category#1058) AND (category#1058 = Electronics))

(3) Project [codegen id : 1]
Output [2]: [product_id#1057, price#1059]
Input [3]: [product_id#1057, category#1058, price#1059]
```

3. Reading Files with Proper Schema Handling

Loading a CSV while treating the first row as a header and letting Spark infer types:

Code:

```
# Read the CSV file into a DataFrame
df = spark.read.csv("../data/orders_data.csv", header=True,
inferSchema=True)

# Display the number of rows and columns in the DataFrame
print("Rows:", df.count(), "| Columns:", len(df.columns))
df.show(7)
```

Output:

```
Rows: 10000 | Columns: 14
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| order_id|product_id| category| price|base_price| amount| status|user_id| region|priority|order_date|quantity|payment_method|old_name|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|ORD100000| P1072| Clothing|1268.94| 1793.83|2123.09| Refunded| U3013| South| Medium|2025-06-19| 5| Credit Card|item_366|
|ORD100001| P1372| Books|3654.49| 1842.62|4088.83| Refunded| U2786| South| Medium|2024-01-25| 1| Debit Card|item_357|
|ORD100002| P1479| Toys|1626.28| 1942.21|2520.27| Cancelled| NULL| East| Low|2024-12-27| 13| Credit Card|item_159|
|ORD100003| P1089| Groceries|3072.87| 1416.06|4873.15| Completed| U3148| North| Low|2025-09-10| 14| UPI|item_245|
|ORD100004| P1842| Clothing|4424.95| 2881.66|1826.08| Completed| U2636| North| Medium|2025-04-29| 11| COD|item_381|
|ORD100005| P1469| Clothing| NULL| 2076.37|1479.06| Cancelled| U2214| East| High|2025-01-10| 4| Credit Card|item_204|
|ORD100006| P1024| Furniture| 2778.0| 2649.05|7758.76| Completed| U2204| Central| High|2024-11-05| 13| COD|item_422|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 7 rows
```

Code:

```
# Displaying the DataFrame schema
df.printSchema()
```

Output:

```
root
|-- order_id: string (nullable = true)
|-- product_id: string (nullable = true)
|-- category: string (nullable = true)
|-- price: double (nullable = true)
|-- base_price: double (nullable = true)
|-- amount: double (nullable = true)
|-- status: string (nullable = true)
|-- user_id: string (nullable = true)
|-- region: string (nullable = true)
|-- priority: string (nullable = true)
|-- order_date: date (nullable = true)
|-- quantity: integer (nullable = true)
|-- payment_method: string (nullable = true)
|-- old_name: string (nullable = true)
```

4. Filtering & Selecting Columns

- **Q5** — pick out product_id and price for rows where the category is 'Electronics':

Code:

```
electronics = df.filter(col("category") ==
"Electronics").select("product_id", "price")
print("Electronics rows:", electronics.count())
electronics.show(7)
```

Output:

```
Electronics rows: 1363
+-----+-----+
|product_id| price|
+-----+-----+
|    P1311|1940.96|
|    P1546|4310.07|
|    P1519|3484.92|
|    P1323|3401.56|
|    P1950|1413.87|
|    P1224| 209.12|
|    P1339|3726.28|
+-----+-----+
only showing top 7 rows
```

Q8 — keep rows whose status is 'Completed' and whose amount exceeds 1000 :

Code:

```
completed = df.filter((col("status") == "Completed") & (col("amount") >
1000))
print("Completed orders over 1000:", completed.count())
completed.select("order_id", "status", "amount").show(7)
```


Output:

Completed orders over 1000: 1777

```
+-----+-----+-----+
| order_id|  status| amount|
+-----+-----+-----+
|ORD100003|Completed|4873.15|
|ORD100004|Completed|1826.08|
|ORD100006|Completed|7758.76|
|ORD100007|Completed|3992.74|
|ORD100009|Completed|3199.83|
|ORD100019|Completed|4742.13|
|ORD100026|Completed|5735.06|
+-----+-----+-----+
only showing top 7 rows
```

Q14 — keep rows where the region is 'North' or the priority is 'High':

Code:

```
north_or_high = df.filter((col("region") == "North") | (col("priority") ==
"High"))
print("North OR High-priority rows:", north_or_high.count())
north_or_high.select("order_id", "region", "priority").show(7)
```

Output:

North OR High-priority rows: 4662

```
+-----+-----+-----+
| order_id| region|priority|
+-----+-----+-----+
|ORD100003|  North|    Low|
|ORD100004|  North|  Medium|
|ORD100005|   East|   High|
|ORD100006|Central|   High|
|ORD100010|  North|  Medium|
|ORD100016|  North|  Medium|
|ORD100018|  South|   High|
+-----+-----+-----+
only showing top 7 rows
```

5. Modifying DataFrames - Rename, Add Columns

- **Q6** — change the column old_name into new_name

Code:

```
df_revised = (df
    .withColumnRenamed("old_name", "new_name")
)

df_revised.select("new_name").show(7)
```

Output:

```
+-----+
|new_name|
+-----+
|item_366|
|item_357|
|item_159|
|item_245|
|item_381|
|item_204|
|item_422|
+-----+
only showing top 7 rows
```

- **Q10** — introduce a `final_price` column equal to `base_price` multiplied by 1.18 (18 percent tax):

Code:

```
df_taxed = df_revised.withColumn("final_price", F.round(col("base_price") *
1.18, 2))
df_taxed.select("base_price", "final_price").show(7)
```

Output:

```
+-----+-----+
|base_price|final_price|
+-----+-----+
|  1793.83|   2116.72|
|  1842.62|   2174.29|
|  1942.21|   2291.81|
|  1416.06|   1670.95|
|  2881.66|   3400.36|
|  2076.37|   2450.12|
|  2649.05|   3125.88|
+-----+-----+
only showing top 7 rows
```

6. Transformations vs Actions

- Transformations are *deferred* — they describe a new DataFrame and return at once without doing any computation. Examples include `filter()`, `select()`, `withColumn()`, and `groupBy()`.
- Actions *set the work in motion*, executing the DAG and either returning a result to the driver or writing output. Examples include `count()`, `show()`, `collect()`, and `write()`.

The snippet below shows a transformation being defined with no work done, followed by an action that kicks off the whole plan:

Code:

```
# Transformation: returns instantly, no job runs
t = df.filter(col("amount") > 5000).select("order_id", "amount")
print("Transformation defined (lazy) - no job has executed yet.")

# Action: triggers computation
print("Action result :", t.count())
```

Output:

```
Transformation defined (lazy) - no job has executed yet.  
Action result : 3780
```

7. Wide Transformations & Performance (Shuffle, Predicate Pushdown)

- Narrow transformations like `filter()`, `select()`, and `withColumn()` operate on data that already sits within one partition. Nothing has to travel between partitions, so these steps tend to be quick.
- Wide transformations like `groupBy()`, `join()`, and `distinct()` need records gathered from many partitions. Rearranging them this way is called a shuffle: data is moved around the cluster and regrouped by key. Since a shuffle relies on network traffic and extra disk activity, it ranks among Spark's costliest operations.

Q7 — Fault tolerance through the lineage graph:

Spark keeps a lineage graph describing every transformation that produced a piece of data. When an executor dies and a partition disappears, Spark uses that lineage to rebuild just the missing partition rather than redoing everything. That is what makes it both resilient and economical.

Q9 — Predicate pushdown:

With columnar formats like Parquet, Spark can push filter conditions down to the read step itself. Only the data that satisfies the filter is pulled off disk, and whole row groups that cannot match are skipped, which trims disk I/O, memory pressure, and runtime.

- The example below runs `groupBy()` with `explain()` to print the plan; the Exchange operator in it marks where a shuffle occurs:

Code:

```
(df.groupBy("region")  
  .agg(F.count("*").alias("orders"),  
       F.round(F.avg("amount"), 2).alias("avg_amount"))  
  .explain())
```

Output:

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[region#1120], functions=[count(1), avg(amount#1117)])
   +- Exchange hashpartitioning(region#1120, 200), ENSURE_REQUIREMENTS,
      [plan_id=1111]
      +- HashAggregate(keys=[region#1120], functions=[partial_count(1),
         partial_avg(amount#1117)])
         +- FileScan csv [amount#1117,region#1120] Batched: false,
            DataFilters: [], Format: CSV, Location: InMemoryFileIndex(1
            paths)[file:/d:/CEI internship/Week
            6/Spark_Assignment/data/orders_data.csv], PartitionFilters: [],
            PushedFilters: [], ReadSchema: struct<amount:double,region:string>
```

Code:

```
# The actual grouped result (a wide transformation)
(df.groupBy("region")
  .agg(F.count("*").alias("orders"),
       F.round(F.avg("amount"), 2).alias("avg_amount"))
  .orderBy(F.col("orders").desc())
  .show())
```

Output:

```
+-----+-----+-----+
| region|orders|avg_amount|
+-----+-----+-----+
|   West|  2084|   4073.57|
|   East|  2020|   4022.63|
|  South|  1993|   3955.69|
|Central|  1952|   4117.12|
|   North| 1951|   4049.41|
+-----+-----+-----+
```

8. CSV vs Parquet - Storage & Performance

Q4 — Row-based versus columnar:

CSV | Parquet

Layout | Row-based — stored one row at a time | Columnar — stored one column at a time

Schema | None (worked out or declared on each read) | Kept inside the file

Compression | Weak (each row mixes types) | Strong (like values sit together per column)

Column pruning | Whole rows must be read | Only requested columns are read

Predicate pushdown | Unavailable | Available (uses row-group statistics)

Why Parquet tends to win: storing rows end to end, as CSV does, is convenient for moving data around but inefficient for heavy analytics. Parquet's column-by-column layout lets Spark grab only the columns a query touches and ignore the rest. The payoff is less disk I/O, lighter memory use, tighter compression, and quicker queries — which is why Parquet is the go-to format in modern data engineering.

9. Handling Null Values & Efficient Filtering

The data carries blank price entries and missing user_ids. The cells below measure how many there are and then deal with them:

Code:

```
from pyspark.sql.functions import when, count

null_report = df.select([
    count(when(col(c).isNull() | (col(c).cast("string") == ""), c)).alias(c)
    for c in df.columns
])

null_report.show(truncate=False)
```

Output:

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|order_id|product_id|category|price|base_price|amount|status|user_id|region|
|priority|order_date|quantity|payment_method|old_name|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
|0        |0        |0        |523  |0        |0        |0        |729    |0
|0        |0        |0        |0     |0        |0        |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
```

- **Q12** — drop every row where user_id is missing:

Code:

```
# Filter out rows with null/empty user_id
df_valid = df.filter(col("user_id").isNotNull() & (col("user_id") != ""))
print("Rows before:", df.count(), "| after removing null user_id:",
      df_valid.count())
```

Output:

```
Rows before: 10000 | after removing null user_id: 9271
```

10 & 11. Build a Pipeline (read → transform → filter → write) and Save

A full pipeline that loads the data, reshapes it (rename, cast, add the tax column), removes invalid rows, and saves the outcome as both Parquet and CSV.

Code:

```
from pyspark.sql.types import (StructType, StructField, StringType,
                                DoubleType, IntegerType)

schema = StructType([
    StructField("order_id",      StringType(), True),
    StructField("product_id",    StringType(), True),
    StructField("category",      StringType(), True),
    StructField("price",         StringType(), True),    # kept as string,
cast later in pipeline
    StructField("base_price",     DoubleType(), True),
    StructField("amount",        DoubleType(), True),
    StructField("status",        StringType(), True),
    StructField("user_id",       StringType(), True),
    StructField("region",        StringType(), True),
    StructField("priority",      StringType(), True),
    StructField("order_date",    StringType(), True),
    StructField("quantity",      IntegerType(), True),
    StructField("payment_method", StringType(), True),
    StructField("old_name",      StringType(), True),
])
```

Code:

```
import shutil

for p in ["pipeline_parquet", "pipeline_csv"]:
    shutil.rmtree(p, ignore_errors=True)

pipeline = (spark.read.csv("../data/orders_data.csv", header=True,
schema=schema) # read
    .withColumnRenamed("old_name", "new_name") #
transform: rename
    .withColumn("price", col("price").cast(DoubleType())) #
transform: cast
    .withColumn("final_price", F.round(col("base_price") * 1.18, 2)) #
transform: add col
    .filter(col("user_id").isNotNull() & (col("user_id") != "")) #
filter: valid users
    .filter(col("status") == "Completed")) #
filter: completed

print("Pipeline output rows:", pipeline.count())
pipeline.select("order_id", "new_name", "price", "final_price",
"status").show(7)

# write: save processed data in both formats
pipeline.write.mode("overwrite").parquet("pipeline_parquet")
pipeline.write.mode("overwrite").csv("pipeline_csv", header=True)
print("Saved pipeline output to Parquet and CSV.")
```

Output:

Pipeline output rows: 1865

```
+-----+-----+-----+-----+-----+
| order_id|new_name|  price|final_price|  status|
+-----+-----+-----+-----+-----+
|ORD100003|item_245|3072.87|    1670.95|Completed|
|ORD100004|item_381|4424.95|    3400.36|Completed|
|ORD100006|item_422| 2778.0|    3125.88|Completed|
|ORD100007|item_427|4779.61|    3665.13|Completed|
|ORD100009|item_214|   NULL|    589.32|Completed|
|ORD100019|item_159|2476.34|    2500.01|Completed|
|ORD100026|item_132| 739.85|    3105.36|Completed|
+-----+-----+-----+-----+-----+
only showing top 7 rows
```

Saved pipeline output to Parquet and CSV.

12. Best Practices for Large Datasets -> show() over collect()

Q15 — Why .show() beats .collect() on a multi-terabyte dataset:

- .collect() gathers every row from the executors and ships them all to the driver. On a very large dataset that can overwhelm the driver's memory and crash the job with an OutOfMemory error.
- .show() brings back only a handful of rows for a quick look. Spark computes just those records, so it is a safe, light way to peek at data without dragging the whole thing into the driver.

NOTE: reach for show(n), take(n), or limit(n) while exploring, and save collect() for cases where you know the result is small.

Code:

```
# Safe: returns only a few rows to the driver
df.select("order_id", "amount").show(5)

# take(n) is also safe – returns a small Python list
sample = df.take(3)
print("take(3) returned", len(sample), "rows to the driver (safe).")
```

Output:

```
+-----+-----+
| order_id| amount|
+-----+-----+
|ORD100000|2123.09|
|ORD100001|4088.83|
|ORD100002|2520.27|
|ORD100003|4873.15|
|ORD100004|1826.08|
+-----+-----+
only showing top 5 rows

take(3) returned 3 rows to the driver (safe).
```

Brief Insights on Performance & Architecture

- Spark runs on a distributed design. The Driver plans the work and schedules it, the Cluster Manager supplies the resources, and the Executors do the processing side by side across the cluster.
- Deferred execution pays off. Because transformations are recorded rather than run on the spot, Spark can tune the entire plan before a single row is touched.
- Lineage keeps things fault tolerant. By remembering how each partition was built, Spark can rebuild only what a failed executor lost instead of starting over.

- Parquet outperforms CSV for analytics. Columnar storage, an embedded schema, compression, and predicate pushdown together cut disk I/O and speed queries up.
- Shuffles carry a real cost. Wide steps such as `groupBy()` and `join()` shift data between partitions, and that movement is one of the biggest influences on how fast a job finishes.
- Watch the driver's memory. Inspect data with `show()`, `take()`, or `limit()`, and steer clear of `collect()` on large datasets unless the result is sure to fit.

Code:

```
# Clean up temporary output directories created during the demo
import shutil
for p in
["out_csv", "out_parquet", "pipeline_parquet", "pipeline_csv", "q12_output_csv"]
:
    shutil.rmtree(p, ignore_errors=True)

spark.stop()
print("Temp outputs cleaned and Spark session stopped.")
```

Output:

```
Temp outputs cleaned and Spark session stopped.
```